Gigzen

Gigzen Private Limited · Bhubaneswar, India
shakhtar-sankur.github.io/gigzen

Two products.
One of them proved the other.

We build software for people who work — and the tooling to prove it holds up before anyone depends on it. One product serves gig drivers across 49 countries. The other is the testing platform we had to build to test the first, and it found five bugs in it that a full manual pass had missed.

PRODUCT 01 · ANDROID, CONSUMER

Waggle

A professional and social home for gig workers. A Swiggy rider, an Uber driver and an Amazon Flex courier are often the same person — no platform connects those identities.

- **Earnings that follow the road.** Money accrues from distance genuinely travelled, so a stationary phone earns nothing and traffic cannot inflate the number.
- **A map with people on it.** Live positions, search any street on earth. Sharing is off until the driver turns it on.
- **Works underground.** Posts and messages written with no signal queue on the device and send on reconnect.
- **Privacy enforced by the database.** Phone numbers are unreadable to other users — not hidden in the UI — and a migration assertion fails if that stops being true.

43 languages, right-to-left included · **61** currencies · **83** countries · **33** gig platforms
React · TypeScript · Capacitor · PostgreSQL — **20** tables under **49** row-level-security policies · **8.2 MB** download · Android 7.0+

Google Play — internal testing Free for workers, permanently.

PRODUCT 02 · DEVELOPER TOOL

Populace

A simulated population that uses your app through its real API, so you can test what needs more than one person — presence, live sync, fan-out, and every permission rule you wrote.

- **One adapter is the whole integration.** Thirteen small methods, two of them required. The engine knows how to be a person and nothing about your product.
- **Real users, front door only.** Real accounts, your API, your permission rules applying. No admin keys, no direct database writes.
- **Honest coverage.** An empty method does not count, so a fresh scaffold reports 2/13 and refuses to run rather than passing while testing nothing.
- **Refuses production three ways.** Environment must declare itself, a denylist is matched against every nested string, and an empty denylist warns loudly. All exit non-zero.

13-method contract · **0** runtime dependencies · **3** production guards · **20/20** self-tests with no backend at all
Node 18+ · per-endpoint p50/p95/p99 · JSON report for CI plus a self-contained HTML page · exits non-zero when problems are found

Proven — taking pilot users AGPL-3.0, source public.

Two runs, three and a half minutes apart. The first is what Populace does for you. The second is what it certifies.

POPULACE REPORT · RUN 1 — 9 AUG 2026

✗ 5 problems found — run failed

5	3m 30s	1
DEFECTS FOUND	TIME TO FIND THEM	BLOCKED ALL SIGNUPS

Against a finished, signed app that had just passed a full manual test of every screen. Signup silently created no profile row for any new user — a privacy fix had made a column unreadable and the write needed to read it. Likes bounced one tap in four. Profile edits failed the same way. Two further faults were in Populace's own reference adapter, one of them an unchecked error.

What this proves about Populace: it finds real defects in software that is already considered done, names the cause in one line, and **exits non-zero** — so the build stays failed until each one is rectified.

POPULACE REPORT · LARGEST CLEAN RUN — 21 AUGUST 2026

✓ No failures across 932,455 API calls

0	932,455	13 / 13
FAILURES	API CALLS	METHODS COVERED

Two hundred simulated drivers across twenty cities in eleven countries, using the backend at the same time for an hour. 2,271 km driven, 81,672 posts, 172,660 likes, 69,064 comments, 76,034 messages, 5,389 group joins. Two hundred accounts created through the real signup path and two hundred deleted through the real deletion path — nothing left behind.

What this proves about Waggle: the shipped build holds up under genuine concurrent multi-user traffic with its own permission rules applying — not under one person tapping through screens.

A later run drove **300 drivers through 1,401,435 calls with zero API failures**, but one call never reached the server — a socket exhausted on the test machine — so it is recorded as inconclusive rather than clean. Three hundred is where throughput stops scaling, **not where Waggle breaks**, which is still unfound. These latencies are loopback and contain no network: the same calls cost about 175 ms against a hosted project. Populace drives two unrelated backends, but both are still ours.

Sankur Kundu — Co-founder & Director, Technology
sankur.kundu.tw@gmail.com

Snehasis Despande — Founder & Director, Business
Gigzen Private Limited, Bhubaneswar, Odisha, India

github.com/Shakhtar-Sankur/waggle
github.com/Shakhtar-Sankur/populace